Trip Planner — Main Project Report

Trip Planner: An AI-Powered Travel Assistant

Abstract

Trip Planner is a browser-based AI travel assistant that turns a few simple inputs — a destination, trip duration, number of travellers, total budget, travel style and interests — into a personalised plan. Rather than a single output, the application produces three connected results from one form: a day-by-day **itinerary**, an estimated **budget breakdown** and a destination-specific **packing checklist**. All three can be saved together as one complete trip and reopened later from the My Trips section.

The interface is organised around a single “Design your journey” card with three tabs (Itinerary, Budget and Packing). The user fills in the form once, chooses what to generate, and reviews the result in a scrollable panel that offers Copy and Clear controls and a Save Complete Trip action that shows which of the three parts have been generated so far.

The application is intentionally positioned as a planning aid. Every generated figure is labelled as an estimate (the budget view states that it is not a live quote), and the tool does not book travel or accommodation. Its value lies in removing the blank-page problem: a traveller gets a structured starting point that can be reviewed, corrected and saved.

Project at a glance

Item	Details
Project name	Trip Planner
Tagline	“Explore more. Plan less.” — your personal AI travel assistant
Outputs	Day-wise itinerary, budget breakdown, packing checklist
Key features	Three-tab generator, Copy result, Clear result, Save Complete Trip, My Trips
Inputs	Destination, duration (days), travellers, total budget (INR), travel style, interests
Frontend / backend / AI / database	Technology details to be confirmed (see Section 3)

How this report is organised

Section	Content
1–2	Project description, objectives, scope and usage scenarios
3–4	Technical overview, prerequisites and configuration
5	Project workflow
6	Application screenshots with explanation
7–9	Testing plan, observations, limitations, future scope and conclusion

1. Project Description

The purpose of Trip Planner is to simplify the first stage of trip planning. A traveller normally has to combine destination research, a rough budget, activity ideas and a packing list from several different sources. Trip Planner brings these together in one interface so that a complete first draft of a trip can be produced in a single sitting.

Problem addressed

Planning a trip usually involves switching between maps, blogs, price comparison pages and note-taking apps, and re-balancing the plan whenever one constraint, such as budget or number of days, changes. Trip Planner takes the key constraints once and generates an itinerary, a budget and a packing list that are consistent with the same inputs.

Project objectives

- Collect only the essential trip details in a short, easy-to-read form.
- Generate a day-by-day itinerary with time-of-day activities, food, transport and cost estimates.
- Generate a category-wise estimated budget and compare it with the traveller's stated budget.
- Generate a packing checklist with the reason and approximate cost for each item.
- Let the user save the itinerary, budget and packing list together as one complete trip.
- Allow earlier plans to be revisited through My Trips.

Target users

Independent travellers, students, couples, small groups and families who want a quick personalised starting plan rather than a fully managed tour. Inputs are limited to common planning details, so the tool is approachable for first-time users.

Scope of the project

In scope	Out of scope (current version)
Short travel-details form	Booking flights, trains, hotels or activities
AI-generated itinerary, budget and packing list	Guaranteed or live prices and availability
Saving the complete trip and viewing it under My Trips	Real-time maps, weather or place ratings (to be confirmed)
Copy and clear controls for generated results	Group collaboration and sharing

Design principles

- **Simplicity first:** one form, one result panel, one saved-trips area.
- **Three views, one input:** itinerary, budget and packing list are generated from the same details.
- **Transparency:** results are marked "Generated with AI" and costs are marked as estimates.
- **User control:** results can be copied, cleared and saved on demand.

2. Usage Scenarios

The following scenarios describe how a traveller is expected to use the application from the first visit to the final saved output.

Scenario 1: Generate an itinerary

A traveller enters *Goa* as the destination, sets the duration to 3 days, travellers to 1, total budget to INR 10,000, travel style to Balanced and interests to “sightseeing, local food, nature”. On pressing **Generate My Itinerary**, the app produces a 3-day plan. Day 1, for example, is titled “North Goa: Beaches & Culture” and is split into morning, afternoon and evening activities with food, transport and cost estimates.

Outcome: a day-wise plan with an estimated total budget, shown in the result panel under the heading “Your personalized plan”.

Scenario 2: Check the estimated budget

The user switches to the Budget tab. The app returns a table of categories (accommodation, food, local transport, activities, miscellaneous and contingency) with an approximate cost and a note for each, followed by a comparison with the budget the user entered. The view states that all figures are estimates and not a live quote.

Outcome: a category-wise estimate the traveller can judge against their own budget.

Scenario 3: Prepare a packing checklist

The user switches to the Packing tab and receives a checklist grouped by category (for example Clothing), where each item has a reason (such as “warm, humid climate”) and an approximate cost.

Outcome: a destination-aware packing list that matches the trip.

Scenario 4: Save and revisit the trip

Once the itinerary, budget and packing list have been generated, the **Save Complete Trip** card shows a tick against each completed part. Pressing the button stores them together and confirms with “Your trip was saved successfully!”. The trip can then be found under **My Trips**.

Outcome: one saved record containing all three parts of the plan.

Scenario summary

Scenario	Trigger	Result
1. Generate itinerary	Generate My Itinerary (Itinerary tab)	Day-wise plan shown
2. Budget estimate	Budget tab	Category-wise estimate shown
3. Packing checklist	Packing tab	Itemised checklist shown
4. Save and revisit	Save Complete Trip / My Trips	Trip stored and reopened

3. Technical Overview

At a functional level the application follows a simple request–response pattern. The user interacts only with the web interface. The interface collects the form values and sends them to an AI generation service, which returns the itinerary, budget or packing text. When the user saves the trip, the three results are stored together so that they can be listed again under My Trips.

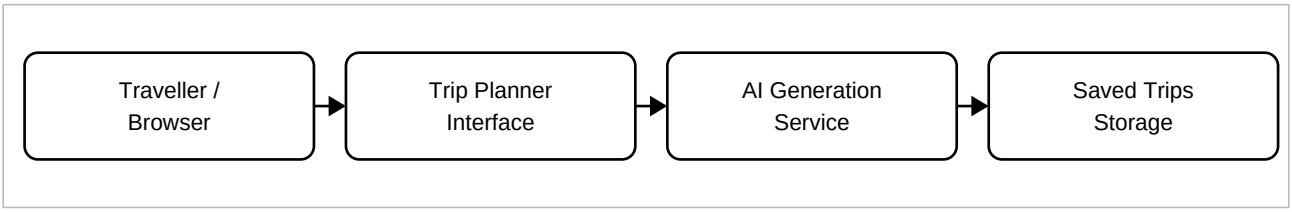


Figure 1. Functional overview of Trip Planner (technology labels to be added).

How the parts interact

The traveller fills in the form and selects Itinerary, Budget or Packing. The interface sends the destination, duration, travellers, budget, style and interests to the AI service. The returned text is displayed in the result panel with Copy and Clear controls. Pressing Save Complete Trip stores the generated parts, and My Trips lists them again later.

Component	Technology	Responsibility
Frontend	[add: e.g. framework / language]	Form, tabs, result panel, Copy / Clear, Save, My Trips
AI service	[add: provider and model]	Generate itinerary, budget and packing content
Backend / API	[add: if any]	Request handling and orchestration
Storage	[add: database or local storage]	Persist saved complete trips
Hosting	[add: platform]	Deploy the application

To confirm: *the technology column above has been left for you to fill in, because the screenshots show the interface but not the implementation. If your stack matches Trip Genius AI (React, FastAPI, Groq, Supabase), you can copy those rows from that report.*

4. Prerequisites and Configuration

The items below are typically needed to run and deploy a project of this kind. Replace or remove rows that do not apply to your implementation.

Prerequisite	Purpose
Runtime and package manager (e.g. Node.js / npm or Python / pip)	Install dependencies and run the application
AI provider API key (private)	Authenticate calls to the AI model that writes the itinerary, budget and packing list
Storage configuration (if used)	Keep saved complete trips
Git and a modern web browser	Clone the repository and view the application
Hosting account	Deploy the application

Configuration values

Secrets such as the AI provider key and any database credentials must stay in environment configuration. They must not be pasted into documentation, screenshots or public source control.

Local setup outline

- Clone the repository and open the project folder.
- Install dependencies and provide the private environment variables.
- Start the application and open the local address in a browser.
- Generate a test itinerary, budget and packing list, then save the trip.

5. Project Workflow

The end-to-end flow from a filled form to a saved trip is shown below, with the purpose of each step.

Step	Action	Purpose
1	The user enters destination, duration, travellers, total budget, travel style and interests.	Capture the preferences that shape the plan
2	The user selects the Itinerary, Budget or Packing tab and presses the generate button.	Choose which output to produce
3	The interface sends the details to the AI service.	Request a draft plan
4	The generated text is displayed in the result panel marked “Generated with AI”.	Present the draft for review
5	The user may Copy the result or Clear it.	Reuse or restart the result
6	The Save Complete Trip card shows which of the three parts are ready.	Show progress toward a complete trip
7	The user presses Save Complete Trip and sees a success message.	Make the plan persistent
8	The saved trip is opened later from My Trips.	Revisit a previous plan

Form fields

Field	Example (from screenshots)	Use in the plan
Where do you want to go?	Goa	Sets the place context
Duration (days)	3	Determines the number of days planned
Travelers	1	Scales cost estimates
Total budget (INR)	10000	Guides costs and activity choices
Travel style	Balanced	Sets the pace and tone
Your interests	Sightseeing, local food, nature	Shapes the type of activities

6. Application Screenshots

6.1 Main Trip Planner interface

The home screen carries the TripPlanner brand, the headline “Explore more. Plan less.” and a short description. The left card, “Design your journey”, contains the three tabs and the travel-details form. The right card, “Your next adventure starts here”, lists the benefits (personalised recommendations, designed around your budget, tailored to your preferences) and a travel tip. A My Trips button sits above the cards.

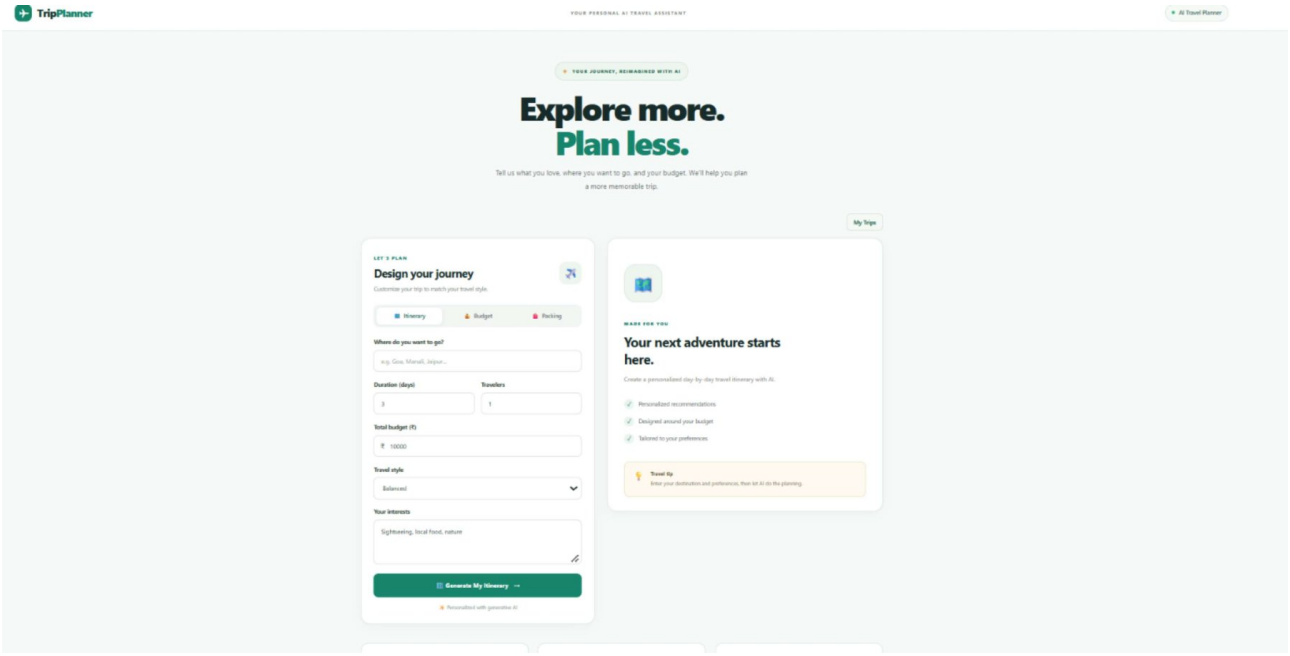


Figure 2. Main Trip Planner interface and travel-details form.

6.2 Generated itinerary

The Itinerary tab shows a result panel titled “Your personalized plan” for the destination, with the trip summary (3 days, 1 traveller, Balanced). The plan opens with a 3-day Goa itinerary, an estimated total budget of INR 10,000 (about US$120) with a note that figures are rough averages, and then Day 1 — North Goa: Beaches & Culture. Each day lists Time, Activity, Highlights, Food, Transport and Cost Estimate. Below the panel, the Save your complete trip card shows the Itinerary part as complete.

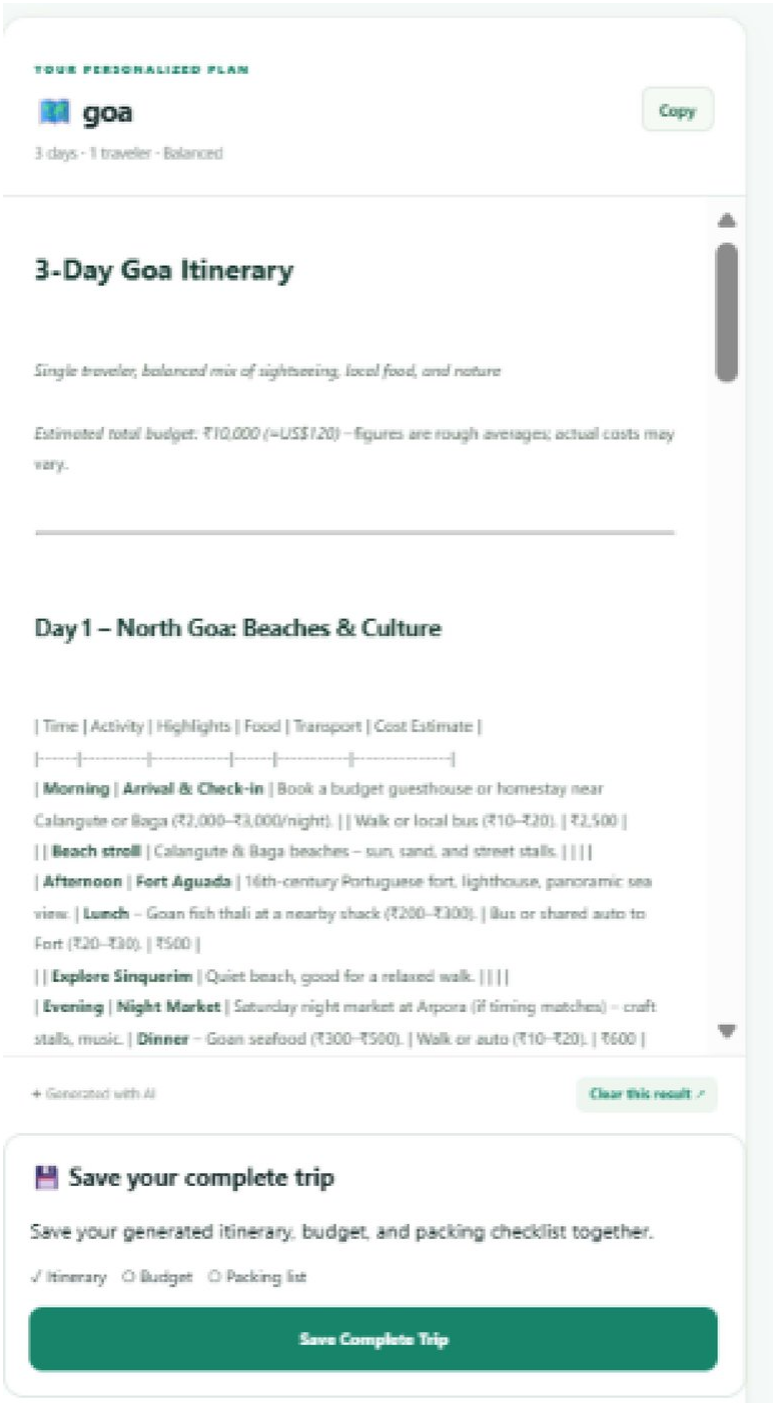


Figure 3. Generated itinerary with day-wise activity suggestions.

6.3 Estimated budget breakdown

The Budget tab shows a practical breakdown with the note that all figures are estimates and not a live quote. Categories are accommodation, food, local transport, activities and sightseeing, miscellaneous and contingency, each with an approximate cost in INR and a note. A “Comparison with your budget” section follows. The save card now shows both Itinerary and Budget as complete.

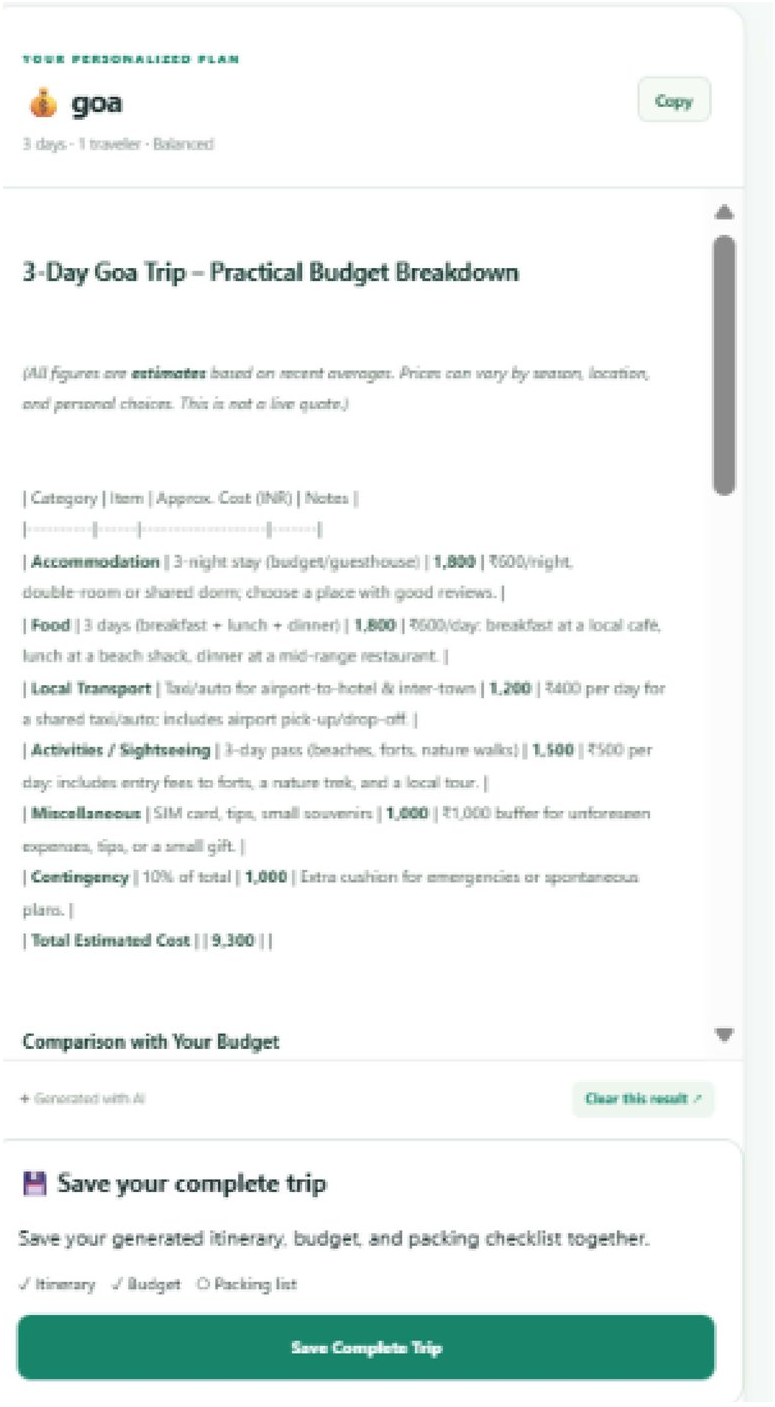


Figure 4. Estimated budget breakdown.

6.4 Packing checklist and save-trip area

The Packing tab produces a 3-day Goa packing checklist grouped by category, starting with Clothing. Each row gives the item, why it is needed and an approximate cost. All three ticks (Itinerary, Budget, Packing list) are shown in the save card, and a banner confirms “Your trip was saved successfully!”.

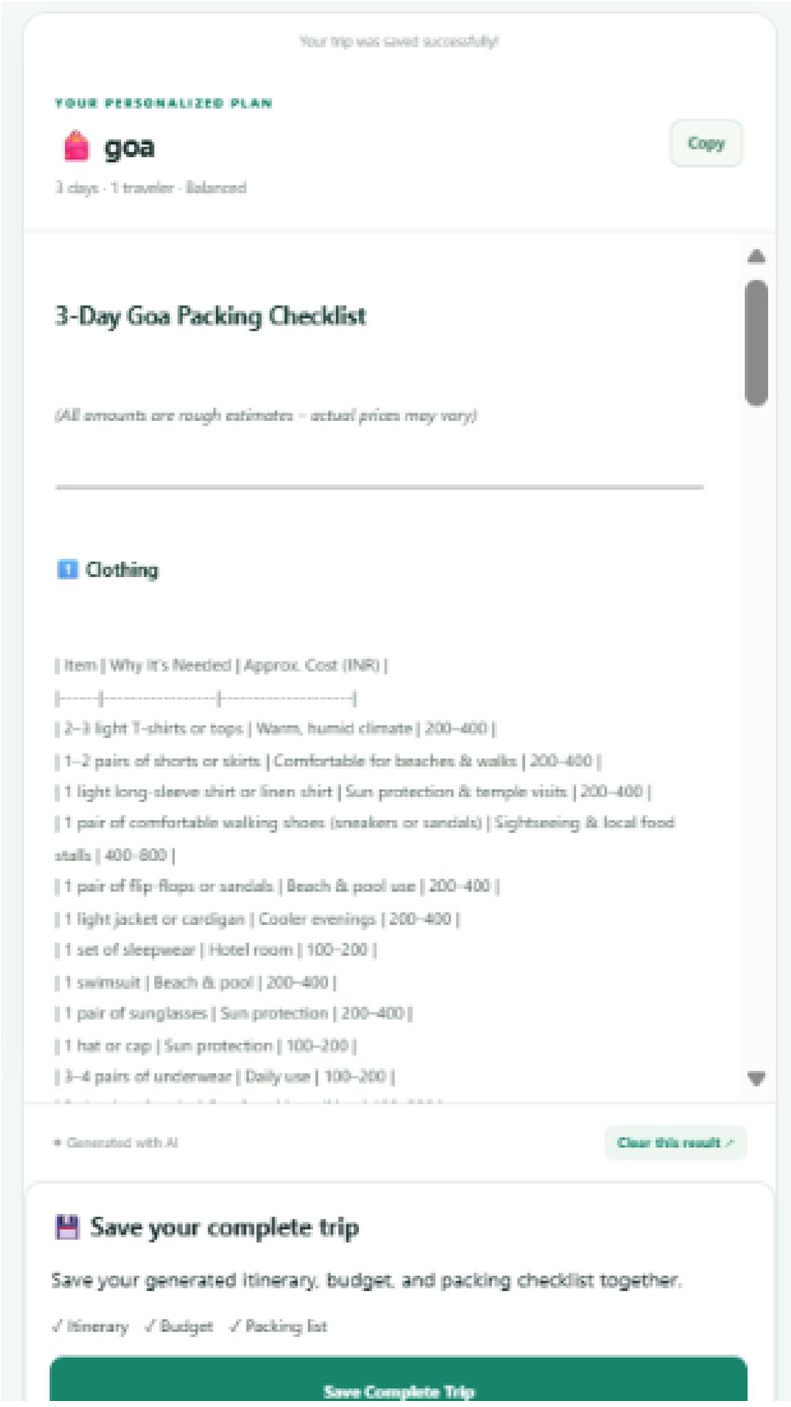


Figure 5. Packing checklist and save-trip area.

7. Testing and Validation Plan

The table lists the checks to be run against the deployed application. Each row states the behaviour that counts as a pass.

Test	Expected behaviour	Status
Application loading	Home page renders without console or build errors	Pending verification
Itinerary generation	Valid inputs return a day-wise plan	Pending verification
Budget generation	Category-wise estimate returned and compared with the entered budget	Pending verification
Packing checklist	Itemised list returned for the destination	Pending verification
Copy and Clear	Copy places the result on the clipboard; Clear empties the panel	Pending verification
Save Complete Trip	Success message appears and the trip is stored	Pending verification
My Trips	Saved trip is listed and opens correctly after reload	Pending verification
Input validation	Empty or invalid fields are handled sensibly	Pending verification

These are planned checks, not claimed results. Execute them against the deployed app and replace each status with the observed outcome. No latency or success-rate values should be reported unless measured.

Observations from the supplied screenshots

Two issues are visible in the supplied evidence and are worth recording as findings, with the fix applied or planned:

- **Budget total does not match its line items.** In the Goa budget screenshot the listed items (1,800 + 1,800 + 1,200 + 1,500 + 1,000 + 1,000) add up to INR 8,300, but the total shown is INR 9,300. The contingency line (“10% of total” = 1,000) also implies a total of INR 10,000. AI-written arithmetic should be checked or computed by the application.
- **Raw table markup is visible.** The itinerary, budget and packing results show text such as pipes and dashes (| Time | Activity |) instead of rendered tables, so the Markdown table output is not being rendered.
- **Cost consistency.** The itinerary suggests guesthouse stays of INR 2,000–3,000 per night, while the budget assumes INR 600 per night. The three outputs are generated separately and can disagree.

8. Limitations, Security and Future Scope

Current limitations

- AI-generated activities and costs are estimates, not quotations.
- The itinerary, budget and packing list can be inconsistent with each other (see Section 7).
- The app does not book transport or accommodation and does not verify availability.
- A generated plan should be reviewed for travel time, season and local conditions.

Security considerations

- Keep API keys and database credentials on the server side only.
- Restrict cross-origin access to the deployed frontend.

- Avoid exposing keys or environment variables in screenshots and logs.
- If trips are stored, decide who may read and delete them.

Future enhancements

- Render Markdown tables properly in all result panels.
- Compute budget totals in code and keep the three outputs consistent.
- User accounts with per-user saved trips.
- Live place, weather and price data; booking integrations.
- Date-based planning, PDF export, multilingual support and automated tests.

9. Deployment and References

Resource	Address
GitHub repository	https://github.com/Khushi1904-lab/trip_planner.git
Demonstration video	https://drive.google.com/file/d/11SZbjVor3PXfOGLrgDt4Z09Xw1K4x3xm/view?usp=sharing

Conclusion

Trip Planner combines three outputs — itinerary, budget and packing checklist — behind one short form and lets the traveller save them together as a complete trip. Its simple interface makes it approachable, and its clear labelling of estimates keeps expectations realistic. With consistent budget calculation, rendered tables, user accounts and live data added in future work, it can grow from a planning assistant into a fuller travel-support tool.

Pre-submission checklist

- Fill in the technology table, repository, live and video links.
- Run the eight tests in Section 7 and update the status column with observed results.
- Fix or document the budget total mismatch and the raw table display.
- Make sure no API keys or .env contents appear in any screenshot.